AIM : To create Multiple Task using free RTOS On an Arduino UNO and understanding the multitasking capabilities of RTOS

THEORY :

A Real-Time Operating System (RTOS) is made to handle many tasks in an embedded system and make sure each task gets the right amount of CPU time.Rather than using one main loop to do everything, an RTOS breaks the work into separate tasks and uses a scheduler to manage when each task runs.

In this experiment, FreeRTOS is used with the Arduino UNO to show how multitasking works.
Two separate tasks are created.The first task shows a message every one second, and the second task shows its message every two seconds.

FreeRTOS gives each task a priority.
The scheduler looks at the status and priority of each task to decide which one should run next.When a task uses vTaskDelay(), it pauses for a set amount of time.During that pause, the processor can run another task that is ready to execute.

This allows different tasks to seem like they are running at the same time.
It also helps organize handling multiple tasks in embedded systems, such as monitoring sensors, sending and receiving data, controlling displays, and managing automation.

OBJECTIVES :

To learn how an RTOS works in general.
To find out how to make separate tasks in FreeRTOS.
To know why task priorities are important.
To use the xTaskCreate function to make tasks.
To use vTaskDelay to manage how often a task runs.
To start the FreeRTOS task manager with vTaskStartScheduler.
To see how tasks run using the Arduino Serial Monitor.

REQUIREMENTS :

Arduino UNO board
USB cable
Computer or laptop
Arduino IDE
Arduino_FreeRTOS library
Serial Monitor
FreeRTOS program

ALGORITHM :

Open the Arduino IDE and connect the Arduino UNO to your computer.
Include the Arduino_FreeRTOS library. The h header file included in the program.
Configure the serial communication to operate at a baud rate of 9600.
Create two distinct functions named task1 and task2 to handle the respective tasks.
Use the xTaskCreate() function to create task1 and set its priority to 1.
Create task2 using the xTaskCreate() function and assign it a priority of 2.
Include the serial number. Include print() commands within each task as necessary.
Use vTaskDelay() to have each task wait for a specified duration.
Call vTaskStartScheduler() to let FreeRTOS manage the tasks.
Check the messages sent by the tasks in the Serial Monitor.

EXPLANATION :

The program starts by using the Arduino FreeRTOS library, which provides the necessary functions for managing tasks.Two separate tasks are created, and each one does a simple job of printing a message.Task1 prints its message and then waits for one second, while Task2 prints its message after waiting for two seconds.

The xTaskCreate() function is used to add both tasks to FreeRTOS.
When setting up a task, you can set details like the task's name, how much memory it needs, its priority, and a handle to identify it.In this case, Task2 has a higher priority than Task1.So if both tasks are ready to run at the same time, the scheduler will start with Task2.

After printing, each task uses vTaskDelay() to pause for a certain amount of time.
This makes the task wait instead of constantly using the CPU.Once the waiting time is done, the task is ready to run again.

The scheduler is started using vTaskStartScheduler().
From this point on, FreeRTOS takes care of running the tasks.Because of this, the loop() function doesn't have any code in it, since the tasks are being managed by the scheduler.

CONCLUSION :

The experiment successfully demonstrates the use of FreeRTOS for managing more than one task on an Arduino UNO.
The experiment therefore provides a practical understanding of how multitasking can be implemented in embedded systems using an RTOS via Free RTOS